ESTRUCTURAS DE DATOS

Árbol B vs Árbol B+

Cómo indexar grandes volúmenes de información en disco

¿Por qué existen los árboles B?

Disco lento

Leer del disco es miles de veces más lento que leer de memoria. Cada acceso cuenta.

Árboles anchos

Cada nodo guarda muchas claves, así el árbol es bajo y se necesitan pocos accesos.

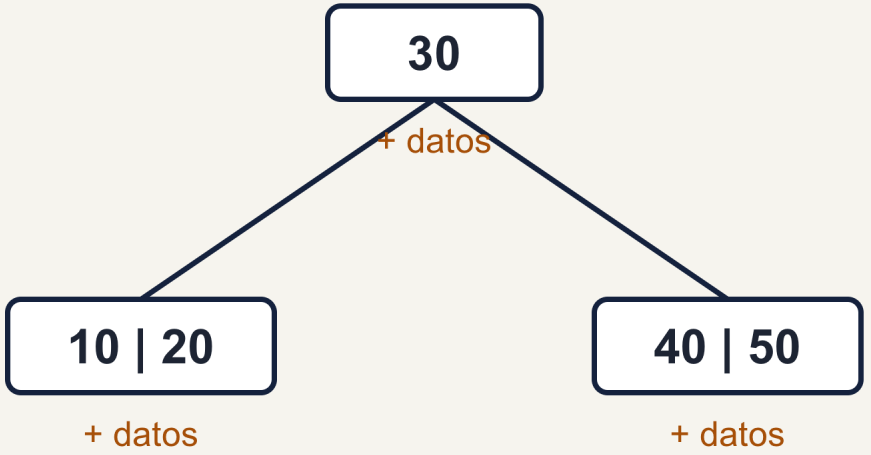
Siempre balanceado

Todas las hojas están al mismo nivel, por lo que buscar, insertar y borrar es O(log n).

Árbol B

Claves y datos en todos los nodos

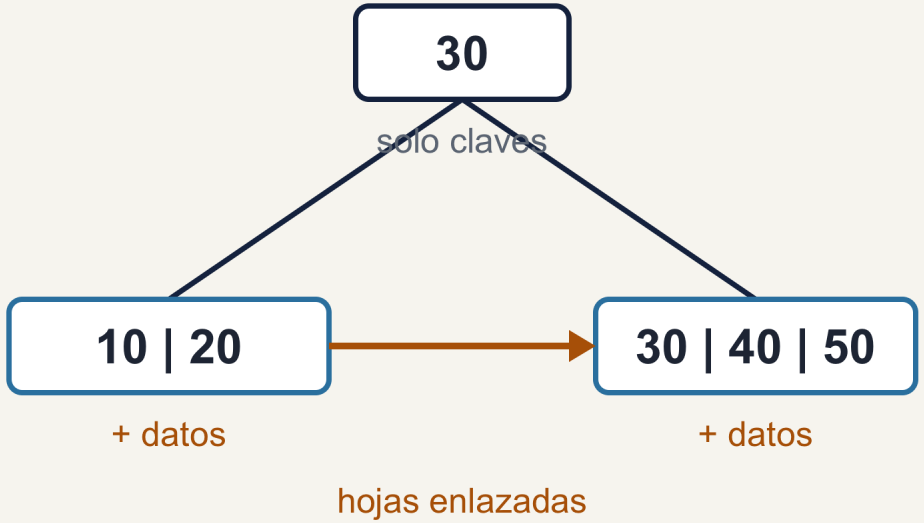
- Cada nodo guarda hasta m−1 claves y m hijos
- Los datos están en nodos internos y en hojas
- La búsqueda puede terminar antes de llegar a una hoja
- Las hojas no están enlazadas entre sí



Árbol B+

Los datos solo viven en las hojas

- Los nodos internos solo guardan claves que guían la búsqueda
- Las claves se repiten: aparecen también en las hojas
- Toda búsqueda baja hasta una hoja
- Las hojas forman una lista enlazada



Estructura del nodo

Árbol B (Python)

```
class NodoB:
    claves = [] # ordenadas
    datos = [] # un dato por clave
    hijos = [] # len(claves) + 1
    es_hoja = False
```

Árbol B+ (Python)

```
class NodoInterno:
    claves = [] # solo guían
    hijos = []

class Hoja:
    claves = []
    datos = [] # los datos viven aquí
    siguiente = None # hoja vecina
```

Búsqueda en árbol B

buscar(nodo, k)

```
def buscar(nodo, k):
    i = 0
    while i < len(nodo.claves) and k > nodo.claves[i]:
        i += 1
    if i < len(nodo.claves) and nodo.claves[i] == k:
        return nodo.datos[i] # puede estar en un nodo interno
    if nodo.es_hoja:
        return None
    return buscar(nodo.hijos[i], k)
```

Búsqueda por rango en árbol B+

rango(raiz, a, b)

```
def rango(raiz, a, b):
    hoja = bajar_hasta_hoja(raiz, a)
    res = []
    while hoja is not None:
        for k, d in zip(hoja.claves, hoja.datos):
            if k > b:
                return res
            if k >= a:
                res.append(d)
        hoja = hoja.siguiente # se recorre la lista enlazada
    return res
```

Comparación

Criterio	Árbol B	Árbol B+
Dónde están los datos	En todos los nodos	Solo en las hojas
Nodos internos	Claves y datos	Solo claves (caben más)
Altura	Algo mayor	Menor
Búsqueda puntual	Puede acabar en un nodo interno	Siempre llega a una hoja
Búsqueda por rango	Lenta: recorrido en orden	Rápida: hojas enlazadas
Claves repetidas	No	Sí, en nodos internos y hojas

¿Cuándo usar cada uno?

Árbol B

- Consultas puntuales por clave
- Pocos recorridos secuenciales
- Interesa encontrar el dato lo antes posible

Árbol B+

- Consultas por rango y ORDER BY
- Índices de bases de datos (MySQL InnoDB, PostgreSQL, SQL Server)
- Sistemas de archivos como NTFS

CONCLUSIÓN

El B+ gana donde importan los rangos y la altura del árbol.

El árbol B sigue siendo la base: simple y eficiente para búsquedas puntuales.